Salus Plugin Backend Protocol

Requirements for the framework implementation

Counterpart: Python backend SDK `salus_sdk.py` (reference implementation and test suite)

Contents

1 Overview 2
2 Transport framing 2
2.1 Frame layout 2
2.2 Validation 2
2.3 Atomic writes 3
2.4 Reference code (Rust) 3
3 Fragmentation 4
4 Addressing 4
5 Logical channels 5
6 `ws://` channels 5
7 `salus://` control protocol 5
7.1 Meta payload container 5
7.2 Requests and replies 6
7.3 Topics 6
7.4 Frontend lifecycle events 7
8 HTTP 7
8.1 Opening and closing routes 7
8.2 Forwarded requests 7
8.3 Responses 8
8.4 Behaviour the SDK guarantees 8
9 Connection lifecycle 9
10 Conformance vectors 9
11 Implementation checklist 10
12 Open decisions 10

1 Overview

Every plugin backend instance talks to the framework over **one** Unix domain stream socket. Everything is multiplexed over that socket: raw socket traffic with the plugin frontends, HTTP requests, and control requests to the framework. The multiplexing key is the **logical channel**, a string carried in every frame.

The framework listens on the socket path it passes to the backend as first command-line argument. The backend connects as a client. There is exactly one connection per backend instance, and several plugin frontends may be bound to the same backend instance.

This document replaces the postcard-encoded `PluginMessageFrame`. The new wire format uses a fixed binary header and has no serde dependency, so that it is trivial to implement in any language.

Conventions. All integers are little-endian. "Must" marks a requirement the framework has to satisfy for the SDK to work. Strings are UTF-8. The Python SDK's behaviour is the reference where this document is silent.

2 Transport framing

2.1 Frame layout

Each frame on the stream is a length prefix followed by the frame body:

Field	Type	Meaning
`frame_len`	`u32`	Number of bytes in the frame body below. Does not include itself.
`frontend_process_id`	`u32`	Inbound (to the framework): the frontend the data comes from, or the reserved id `0`. Outbound (to the backend): the target frontend, or a reserved id. See Section 4.
`message_id`	`u32`	Identifies a message among those in flight from the same sender. See Section 3.
`flags`	`u8`	Bit 0 (`0x01`) is `MORE_FRAGMENTS`. All other bits are reserved and must be zero.
`channel_len`	`u16`	Length of the logical channel in bytes.
`logical_channel`	bytes	`channel_len` bytes of UTF-8.
`payload`	bytes	Remainder of the frame body. No length field of its own.

The fixed header is 11 bytes (`4 + 4 + 1 + 2`). Frame body length is `11 + channel_len + payload_len`.

Limits. `frame_len` must not exceed `67108864` (64 MiB). The logical channel must not exceed 65535 bytes.

2.2 Validation

On receiving a frame the framework must handle errors as the SDK does:

Condition	Action
`frame_len` greater than 64 MiB	Fatal for the connection. Close it, the stream cannot be resynchronised.
Connection closes inside a length prefix or inside a frame body	Treat as an error close of the connection.
Body shorter than 11 bytes, or `channel_len` runs past the end of the body	Drop the frame, log a warning, continue reading.
Channel is not valid UTF-8	Drop the frame, log a warning, continue reading.
Reserved flag bits set	Drop the frame, log a warning, continue reading.

When sending, the framework must not emit frames violating these rules.

2.3 Atomic writes

Frames from different logical sources are interleaved on the same stream. The framework must serialise writes **per frame**: the length prefix and the body of one frame must be written without any bytes of another frame in between. The existing per-stream mutex is sufficient, provided it is held for the whole frame and not just for the whole message (see Section 3).

2.4 Reference code (Rust)

```
pub const HEADER_LEN: usize = 11;
pub const MAX_FRAME_SIZE: usize = 64 * 1024 * 1024;
pub const FLAG_MORE_FRAGMENTS: u8 = 0x01;
pub const NO_FRONTEND: u32 = 0;
pub const ALL_FRONTENDS: u32 = u32::MAX;

pub struct PluginMessageFrame {
    pub frontend_process_id: u32,
    pub message_id: u32,
    pub flags: u8,
    pub logical_channel: String,
    pub payload: Vec<u8>,
}

fn invalid(msg: &str) -> std::io::Error {
    std::io::Error::new(std::io::ErrorKind::InvalidData, msg.to_string())
}

impl PluginMessageFrame {
    /// Body only, without the u32 length prefix.
    pub fn encode(&self) -> std::io::Result<Vec<u8>> {
        let channel = self.logical_channel.as_bytes();
        if channel.len() > u16::MAX as usize { return Err(invalid("channel too
long")); }
        if self.flags & !FLAG_MORE_FRAGMENTS != 0 { return Err(invalid("unknown
flags")); }
        let len = HEADER_LEN + channel.len() + self.payload.len();
        if len > MAX_FRAME_SIZE { return Err(invalid("frame too large")); }

        let mut b = Vec::with_capacity(len);
        b.extend_from_slice(&self.frontend_process_id.to_le_bytes());
        b.extend_from_slice(&self.message_id.to_le_bytes());
        b.push(self.flags);
        b.extend_from_slice(&(channel.len() as u16).to_le_bytes());
        b.extend_from_slice(channel);
        b.extend_from_slice(&self.payload);
        Ok(b)
    }

    pub fn decode(body: &[u8]) -> std::io::Result<Self> {
        if body.len() < HEADER_LEN { return Err(invalid("frame shorter than header")); }
        let flags = body[8];
        if flags & !FLAG_MORE_FRAGMENTS != 0 { return Err(invalid("unknown flags")); }
        let channel_len = u16::from_le_bytes([body[9], body[10]]) as usize;
        let channel_end = HEADER_LEN + channel_len;
        if channel_end > body.len() { return Err(invalid("channel past end of
frame")); }
        let logical_channel = std::str::from_utf8(&body[HEADER_LEN..channel_end])
            .map_err(|_| invalid("channel is not UTF-8"))?
            .to_string();
        Ok(Self {
```

```
            frontend_process_id: u32::from_le_bytes(body[0..4].try_into().unwrap()),
            message_id: u32::from_le_bytes(body[4..8].try_into().unwrap()),
            flags,
            logical_channel,
            payload: body[channel_end..].to_vec(),
        })
    }
}
```

3 Fragmentation

A single frame carries at most 64 MiB. Larger messages are split into several frames, each of which is a complete, valid frame.

Rule	Detail
Identity	All fragments of one message carry the same `frontend_process_id`, `message_id` and `logical_channel`.
Flags	Every fragment except the last has `MORE_FRAGMENTS` set. The last fragment has it cleared. A message that fits one frame is a single frame with the flag cleared.
Order	Fragments of one message appear on the stream in order. Frames of **different** messages may be interleaved between them. This lets small messages overtake a large transfer.
Reassembly key	`(frontend_process_id, message_id)`. A message becomes complete, and is delivered, when its last fragment arrives. Complete messages are therefore delivered in order of completion, not order of start.
`message_id` allocation	Chosen by each sender independently. A wrapping `u32` counter is sufficient. An id must not be reused while a fragmented message with that id is still in flight.
Channel consistency	A fragment whose channel differs from the first fragment is a protocol violation: drop the partial message, log, and discard the remaining fragments of that key until its last fragment has arrived.
Size cap	The framework must cap the reassembled size of a message (the SDK uses 1 GiB) and treat exceeding it as the violation above.

What the framework must implement.

- **Receiving from the backend.** Reassemble fragmented messages before interpreting or forwarding them. This applies to all channel types, including large `http://` responses.
- **Sending to the backend.** If a message exceeds `64 MiB - 11 - channel_len` bytes of payload, split it into fragments as above. Smaller messages must not be fragmented unnecessarily.
- **The SDK itself** only fragments when a message does not fit one frame, so the common case is a single frame with `flags = 0`.

The `message_id` has no meaning beyond fragment reassembly. It is unrelated to the request ids used by `salus://` and `http://` (those live inside the meta payload).

4 Addressing

`frontend_process_id` has a direction-dependent meaning:

Direction	Meaning
Frontend to backend	The sender: the frontend process that originated the data.
Backend to frontend	The target frontend process.

Two values are reserved:

Value	Name	Use
`0`	`NO_FRONTEND`	Used on `salus://` frames: backend to framework requests, and the framework's replies. It means "no frontend involved".
`4294967295` (`0xFFFFFFFF`)	`ALL_FRONTENDS`	Broadcast. When a backend sends a frame with this target, the framework must deliver it to **every** frontend currently bound to this backend instance. It is only meaningful for `ws://` frames.

The framework therefore must keep, per backend instance, the set of bound frontends, and must resolve the backend instance from the frontend for inbound traffic (as `get_be_pid_for_fe_pid` does today). It must verify that outbound targets are bound to this backend and must drop and log frames to other frontends.

5 Logical channels

The logical channel is a UTF-8 string. Its prefix selects the protocol:

Prefix	Section	Summary
`ws://`	Section 6	Raw bidirectional messages between a frontend and the backend. Payload is opaque.
`http://`	Section 8	HTTP requests forwarded to the backend, and the backend's responses.
`salus://`	Section 7	Control requests from the backend to the framework, and framework events.

Frames with any other prefix must be forwarded unchanged where a target is clear and must not be treated as an error. The SDK exposes them to the plugin author as unclaimed messages.

6 `ws://` channels

`ws://<name>` carries opaque bytes. The payload format is entirely defined by plugin author and frontend.

- The framework forwards `ws://` frames between frontend and backend **unchanged** (same channel, same payload). It must not interpret or validate channel names, and there is no registration step for them.
- Frontend to backend: set `frontend_process_id` to the sending frontend.
- Backend to frontend: deliver to the frontend in `frontend_process_id`, or to all bound frontends for `ALL_FRONTENDS`.
- The backend drops (with a warning) messages for channels it has no handler for. The framework does not need to track this.

7 `salus://` control protocol

The backend uses `salus://<topic>` to send requests to the framework and the framework replies on the same channel. The framework also sends unsolicited events.

7.1 Meta payload container

All `http://` and `salus://` payloads use this layout:

Field	Type	Meaning
`meta_len`	`u32`	Length of the meta in bytes.
`meta`	bytes	UTF-8 JSON text of a JSON **object**.
`body`	bytes	Remainder of the payload. Raw bytes, may be empty.

JSON requirements:

- The meta must be a JSON object (not an array or scalar). It must never be empty, so an "empty" meta is the two bytes `{}` (`meta_len = 2`), not `meta_len = 0`.
- `NaN` and `Infinity` are not valid. Key order and whitespace are not significant.
- The framework must **ignore unknown fields** in meta it receives, so that the SDK can add fields later.

- A payload shorter than 4 bytes, a `meta_len` running past the payload, invalid JSON or a non-object meta is a malformed message: drop it and log a warning.

```rust
pub fn pack_meta<T: serde::Serialize>(meta: &T, body: &[u8]) ->
serde_json::Result<Vec<u8>> {
    let json = serde_json::to_vec(meta)?;
    let mut out = Vec::with_capacity(4 + json.len() + body.len());
    out.extend_from_slice(&(json.len() as u32).to_le_bytes());
    out.extend_from_slice(&json);
    out.extend_from_slice(body);
    Ok(out)
}

pub fn unpack_meta<T: serde::de::DeserializeOwned>(payload: &[u8]) -> Result<(T, &[u8]),
String> {
    if payload.len() < 4 { return Err("payload shorter than meta length
prefix".into()); }
    let meta_len = u32::from_le_bytes(payload[0..4].try_into().unwrap()) as usize;
    let end = 4usize.checked_add(meta_len).filter(|e| *e <= payload.len())
        .ok_or("meta runs past end of payload")?;
    let meta = serde_json::from_slice(&payload[4..end]).map_err(|e| e.to_string())?;
    Ok((meta, &payload[end..]))
}
```

7.2 Requests and replies

A request is sent by the backend on `salus://<topic>` with `frontend_process_id = 0`. The meta carries an integer `id` chosen by the backend, plus topic-specific parameters. The framework must answer **every** request on the **same channel** with `frontend_process_id = 0` and:

Field	Type	Meaning
`id`	integer	Copied from the request.
`ok`	boolean	`true` on success, `false` on failure.
`error`	string, optional	Human-readable reason when `ok` is `false`. Shown to the plugin author as `SalusError`.
others	any	Topic-specific result fields. Optional. The reply body is optional too.

Requirements:

- **Every** request gets a reply, including unknown topics (reply `ok: false`, `error: "unknown topic"`) and malformed requests that still carry an `id`.
- The SDK waits **5 seconds** for the reply and then raises a timeout. The framework should reply well within this.
- Replies may be sent in any order. The backend may have many requests in flight.
- The `id` is unique per backend connection while the request is in flight. The framework only echoes it and must not interpret it.
- The key `id` is reserved in request meta. Topic parameters never use it.

7.3 Topics

Topic	Direction	Meaning
`salus://http/open`	backend to framework	Open an HTTP route. See Section 8.
`salus://http/close`	backend to framework	Close an HTTP route. See Section 8.
`salus://frontend/attached`	framework to backend	Event: a frontend was bound to this backend.
`salus://frontend/detached`	framework to backend	Event: a frontend was unbound.

7.4 Frontend lifecycle events

The framework sends these as unsolicited frames. The affected frontend is in the **envelope's** `frontend_process_id`. The meta is the empty object `{}` and there is no body:

```
frontend_process_id = <frontend>   channel = "salus://frontend/attached"
payload = 02 00 00 00 7b 7d        (meta_len = 2, meta = "{}")
```

- Events carry no `id` and need no reply.
- Immediately after accepting a backend connection, the framework must send an `attached` event for **every frontend already bound** to that backend instance. The SDK builds its set of bound frontends solely from these events.
- When a frontend unbinds, send `detached`.

8 HTTP

Plugin backends can serve HTTP-style requests from their frontends. The framework owns the HTTP side: it receives requests from frontends, matches them against the routes the backend opened, forwards them over the socket, and turns the backend's response back into an HTTP response.

8.1 Opening and closing routes

Requests on `salus://http/open` and `salus://http/close`, with this meta (plus `id`):

Field	Type	Meaning
`method`	string	One of `GET`, `POST`, `PUT`, `PATCH`, `DELETE`, `HEAD`, `OPTIONS`, uppercase.
`path`	string	Route pattern. Starts with `/`. Relative to the plugin: the framework namespaces it per plugin, the SDK never sees the prefix.

Pattern syntax. The path is split on `/` into segments. A literal segment must match exactly. A segment of the form `{name}` matches exactly one non-empty request segment and captures it as parameter `name`. Where several routes match, a literal segment wins over a parameter. The `matchit` crate (`{id}` syntax) implements these rules.

Behaviour.

- The set of routes is per `(plugin backend instance, method, path)`.
- Opening a route that is already open, an invalid pattern, or a pattern that conflicts with an existing one (for example `/doc/{id}` and `/doc/{name}` for the same method) must be rejected with `ok: false` and an `error` text.
- Closing a route that is not open must be rejected with `ok: false`.
- After a successful `close` the framework must stop forwarding requests for that route.
- When the backend connection ends, the framework must drop all routes opened over it.
- The SDK opens its statically declared routes right after connecting, before it starts the plugin's own code. The framework must therefore accept `salus://http/open` as soon as the backend connection is accepted.

8.2 Forwarded requests

For a request that matches an open route, the framework sends a frame to the backend:

- **Channel:** any string starting with `http://`. `http://request` is recommended. The SDK identifies requests by their meta, not by the rest of the channel.
- **`frontend_process_id`:** the frontend that made the request.
- **Payload:** the meta container (Section 7.1), with the raw request body as `body`.

Meta field	Type	Required	Meaning
`id`	integer	yes	Request id chosen by the framework. Unique among in-flight HTTP requests of

Meta field	Type	Required	Meaning
			this backend connection. Echoed in the response.
`method`	string	yes	HTTP method.
`path`	string	yes	The concrete request path, relative to the plugin, for example `/doc/42`.
`route`	string	yes	The route pattern that matched, exactly as the backend registered it, for example `/doc/{id}`. The SDK looks up its handler by `(method, route)`.
`params`	object of strings	no, default `{}`	Captured parameters, for example `{"id": "42"}`. Values are percent-decoded.
`query_string`	string	no, default `""`	Raw query string without the leading `?`, as received. The SDK parses it.
`headers`	array of `[name, value]` pairs	no, default `[]`	Request headers. An array of pairs rather than an object, so repeated headers survive.

The framework decides which request headers to forward.

8.3 Responses

The backend answers each request with a frame:

- **Channel:** `http://response`.
- **`frontend_process_id`:** the frontend the request came from.
- **Payload:** the meta container with the raw response body as `body`.

Meta field	Type	Meaning
`id`	integer	The `id` of the request this answers.
`status`	integer	HTTP status, 100 to 599.
`headers`	array of `[name, value]` pairs	Response headers. The SDK adds `content-type` for its helpers.

The framework must:

- Match the response to the pending request by `id` and complete the HTTP response for the frontend. Responses with unknown or already completed `id` are ignored (log a warning).
- Support response bodies larger than 64 MiB by reassembling fragments (Section 3).
- Apply its own policy for requests the backend never answers (a timeout such as 504 is recommended) and for a backend that disconnects while requests are pending (502 is recommended). The SDK does not time out handlers.

8.4 Behaviour the SDK guarantees

The framework does not need to implement these, only relay the results:

- A request for a route the backend no longer has gets a `404` response from the SDK. A malformed request that carries an `id` gets `400`.
- A handler that fails, or returns something that cannot be sent, results in a `500` response.
- A request that lacks an `id` cannot be answered and is dropped by the SDK. The framework must always include one.

9 Connection lifecycle

Event	Required behaviour
Backend connects	Accept. Send `salus://frontend/attached` for each frontend already bound. Be ready for `salus://http/open` requests immediately.
Frontend binds / unbinds while connected	Send the `attached` / `detached` event.
Backend closes the socket	Fail pending HTTP requests, drop its routes, drop partial fragmented messages.
Framework closes the socket	The SDK treats this as shutdown: handlers already running get 5 seconds to finish, then the backend exits.
Backend sends garbage	See the validation table. Only a bad length prefix or a truncated frame is fatal.

10 Conformance vectors

Hex is shown with the 4-byte length prefix included, as it appears on the stream. These come from the SDK's own encoder and are also checked in its test suite.

V1. Plain `ws://` message. Frontend `1`, message id `2`, channel `ws://a`, payload `hi`:

```
13000000 01000000 02000000 00 0600 77733a2f2f61 6869
```

V2. One message in three fragments. Frontend `3`, message id `5`, channel `ws://big`, payload `0123456789` split as `0123`, `4567`, `89`:

```
17000000 03000000 05000000 01 0800 77733a2f2f626967 30313233
17000000 03000000 05000000 01 0800 77733a2f2f626967 34353637
15000000 03000000 05000000 00 0800 77733a2f2f626967 3839
```

The first two have `MORE_FRAGMENTS` set, the last does not. Another message may be interleaved between any two of these frames. It must not be interleaved *inside* one.

V3. Frontend attached event. Frontend `12`, message id `3`, channel `salus://frontend/attached`, meta `{}`:

```
2a000000 0c000000 03000000 00 1900 73616c75733a2f2f66726f6e74656e642f6174746163686564
020000007b7d
```

V4. Route open and its reply. Backend to framework, `frontend_process_id = 0`:

```
channel  salus://http/open
meta     {"id":1,"method":"GET","path":"/doc/{id}"}
body     (empty)
```

Framework to backend, same channel, `frontend_process_id = 0`:

```
channel  salus://http/open
meta     {"id":1,"ok":true}
```

V5. A forwarded request and its response. Framework to backend, from frontend `12`:

```
channel  http://request
meta     {"id":7,"method":"POST","path":"/doc/42","route":"/doc/{id}",
          "params":{"id":"42"},"query_string":"a=1",
```

```
          "headers":[["content-type","text/plain"]]}
body      hello
```

Backend to framework, to frontend 12:

```
channel   http://response
meta      {"id":7,"status":200,"headers":[["content-type","text/plain"]]}
body      ok
```

11 Implementation checklist

#	Item
1	Replace postcard `PluginMessageFrame` with the fixed 11-byte header codec. Keep the `u32` length prefix.
2	Enforce the 64 MiB frame limit, UTF-8 channels, and the validation table. Drop bad frames, close on bad length prefix or truncation.
3	Hold the stream write lock per frame so frames never interleave byte-wise.
4	Reassemble fragmented messages from the backend by `(frontend_process_id, message_id)` with a size cap. Fragment messages above 64 MiB to the backend.
5	Implement reserved ids: `0` for the framework, `0xFFFFFFFF` as broadcast to all frontends bound to the backend.
6	Forward `ws://` frames unchanged in both directions without interpreting channel names.
7	Implement the meta container (`u32` length, JSON object, raw body). Ignore unknown meta fields.
8	Implement `salus://` request handling: reply to **every** request, same channel, `id` echoed, `ok` / `error`, within 5 seconds.
9	Send `salus://frontend/attached` for already-bound frontends on connect, then on every bind. Send `detached` on unbind.
10	Implement `salus://http/open` and `close`: per-plugin namespaced route table, `{name}` single-segment patterns, literal beats parameter, reject duplicates and conflicts.
11	Match incoming frontend HTTP requests, forward them as `http://` frames with `id`, `method`, `path`, `route`, `params`, `query_string`, `headers`, raw body.
12	Accept `http://response` frames and complete the pending HTTP request by `id`. Time out unanswered requests and fail them when the backend disconnects.
13	Clean up on backend disconnect: routes, pending requests, partial messages.
14	Update the Rust test backend (`src/main.rs`) to the new frame format. Run the conformance vectors as unit tests on both sides.

12 Open decisions

These are assumptions made in the SDK that the framework implementer should confirm or change (the SDK is a small change if so):

Topic	Current assumption
Reserved ids	`0` is never a real frontend process id, and `0xFFFFFFFF` is never either.
HTTP request channel name	`http://request` is recommended, but the SDK accepts any `http://` channel for requests.
HTTP timeouts	The framework applies them. The SDK has none for handlers.
Header policy	The framework chooses which request headers to forward and which response headers to pass on (for example hop-by-hop headers).

Topic	Current assumption
Reassembly cap	1 GiB per message (SDK). The framework may choose another limit.
Version handshake	None. A `salus://hello` exchange carrying the protocol version could be added later without changing the frame format.